ChatGPT

Sui GraphQL Beta Schema Reference

Operations

Directives (GraphQL Query Directives)

include

No description

```
directive @include(
  if: Boolean!
) on
  | FIELD
  | FRAGMENT_SPREAD
  | INLINE_FRAGMENT
```

Arguments:

- **include.if ● Boolean!** (non-null scalar)

skip

No description

```
directive @skip(
  if: Boolean!
) on
  | FIELD
  | FRAGMENT_SPREAD
  | INLINE_FRAGMENT
```

Arguments:

- **skip.if ● Boolean!** (non-null scalar)

Mutations

executeTransaction

Execute a transaction, committing its effects on chain.

- `transactionDataBcs` contains the BCS-encoded transaction data (Base64-encoded).
- `signatures` are a list of `flag || signature || pubkey` bytes, Base64-encoded.

Waits until the transaction has reached finality on chain to return its transaction digest, or returns the error that prevented finality if that was not possible. A transaction is final when its effects are guaranteed on chain (it cannot be revoked).

There may be a delay between transaction finality and when GraphQL requests (including the request that issued the transaction) reflect its effects. As a result, queries that depend on indexing the state of the chain (e.g. contents of output objects, address-level balance information at the time of the transaction), must wait for indexing to catch up by polling for the transaction digest using `Query.transaction`.

```
executeTransaction(
  transactionDataBcs: Base64!
  signatures: [Base64!]!
): ExecutionResult!
```

Arguments:

- **executeTransaction.transactionDataBcs ● Base64!** (non-null scalar)
- **executeTransaction.signatures ● [Base64!]!** (non-null scalar list)

Type: ExecutionResult! (object) – The execution result of a transaction, including the transaction effects and any potential errors due to signing or quorum-driving [1] [2].

Queries

address

Look-up an account by its SuiAddress.

If `rootVersion` is specified, nested dynamic field accesses will be fetched at or before this version. This can be used to fetch a child or ancestor object bounded by its root object's version, when its immediate parent is wrapped, or a value in a dynamic object field. For any wrapped or child (object-owned) object, its root object can be defined recursively as:

- The root object of the object it is wrapped in, if it is wrapped.
- The root object of its owner, if it is owned by another object.
- The object itself, if it is not object-owned or wrapped.

Specifying a `rootVersion` disables nested queries for paginating owned objects or dynamic fields (these queries are only supported at checkpoint boundaries).

```
address(
  address: SuiAddress!
  rootVersion: UInt53
): Address!
```

Arguments:

- **address.address ● SuiAddress!** (non-null scalar)
- **address.rootVersion ● UInt53** (scalar)

Type: Address! (object) [3] [4]

chainIdentifier

First four bytes of the network's genesis checkpoint digest (uniquely identifies the network), hex-encoded.

```
chainIdentifier: String!
```

Type: String! (scalar) – The `String` scalar type represents textual data, represented as UTF-8 character sequences [5].

checkpoint

Fetch a checkpoint by its sequence number, or the latest checkpoint if no sequence number is provided.

Returns `null` if the checkpoint does not exist in the store, either because it never existed or because it was pruned.

```
checkpoint(
  sequenceNumber: UInt53
): Checkpoint
```

Arguments:

- **checkpoint.sequenceNumber ● UInt53** (scalar)

Type: Checkpoint (object) [6] [7]

checkpoints

Paginate checkpoints in the network, optionally bounded to checkpoints in the given epoch.

```
checkpoints(
  first: Int
  after: String
  last: Int
  before: String
  filter: CheckpointFilter
): CheckpointConnection
```

Arguments:

- **checkpoints.first ● Int** (scalar)
- **checkpoints.after ● String** (scalar)
- **checkpoints.last ● Int** (scalar)
- **checkpoints.before ● String** (scalar)
- **checkpoints.filter ● CheckpointFilter** (input object)

Type: CheckpointConnection (object) 8 9

coinMetadata

Fetch the CoinMetadata for a given coin type.

Returns `null` if no CoinMetadata object exists for the given coin type.

```
coinMetadata(
  coinType: String!
): CoinMetadata
```

Arguments:

- **coinMetadata.coinType ● String!** (non-null scalar)

Type: CoinMetadata (object) 10 11

epoch

Fetch an epoch by its ID, or fetch the latest epoch if no ID is provided.

Returns `null` if the epoch does not exist yet, or was pruned.

```
epoch(
  epochId: UInt53
): Epoch
```

Arguments:

- **epoch.epochId ● UInt53** (scalar)

Type: Epoch (object) – Activity on Sui is partitioned in time, into epochs. Epoch changes are opportunities for the network to reconfigure itself (perform protocol or system package upgrades, or change the committee) and distribute staking rewards. The network aims to keep epochs roughly the same duration as each other. During a particular epoch the following data is fixed: (protocol version, reference gas price, system package versions, validators in the committee) [12] [13] .

epochs

Paginate epochs that are in the network.

```
epochs(
  first: Int
  after: String
  last: Int
  before: String
): EpochConnection
```

Arguments:

- **epochs.first ● Int** (scalar)
- **epochs.after ● String** (scalar)
- **epochs.last ● Int** (scalar)
- **epochs.before ● String** (scalar)

Type: EpochConnection (object) [14] [15]

events

Paginate events that are emitted in the network, optionally filtered by event filters.

```
events(
  first: Int
  after: String
  last: Int
  before: String
  filter: EventFilter
): EventConnection
```

Arguments:

- **events.first ● Int** (scalar)

- **events.after** ● **String** (scalar)
- **events.last** ● **Int** (scalar)
- **events.before** ● **String** (scalar)
- **events.filter** ● **EventFilter** (input object)

Type: EventConnection (object) 16 17

multiGetCheckpoints

Fetch checkpoints by their sequence numbers.

Returns a list of checkpoints that is guaranteed to be the same length as `keys`. If a checkpoint in `keys` could not be found in the store, its corresponding entry in the result will be `null`. This could be because the checkpoint does not exist yet, or because it was pruned.

```
multiGetCheckpoints(
  keys: [UInt53!]!
): [Checkpoint]!
```

Arguments:

- **multiGetCheckpoints.keys** ● **[UInt53!]!** (non-null list of scalar)

Type: [Checkpoint]! (list of object) 18 19

multiGetEpochs

Fetch epochs by their IDs.

Returns a list of epochs that is guaranteed to be the same length as `keys`. If an epoch in `keys` could not be found in the store, its corresponding entry in the result will be `null`. This could be because the epoch does not exist yet, or because it was pruned.

```
multiGetEpochs(
  keys: [UInt53!]!
): [Epoch]!
```

Arguments:

- **multiGetEpochs.keys** ● **[UInt53!]!** (non-null list of scalar)

Type: [Epoch]! (list of object) 20 21

multiGetObjects

Fetch objects by their keys.

Returns a list of objects that is guaranteed to be the same length as `keys`. If an object in `keys` could not be found in the store, its corresponding entry in the result will be `null`. This could be because the object never existed, or because it was pruned.

```
multiGetObjects(
  keys: [ObjectKey!]!
): [Object]!
```

Arguments:

- **multiGetObjects.keys ● [ObjectKey!]!** (non-null list of input object)

Type: [Object]! (list of object) 22 23

multiGetPackages

Fetch packages by their keys.

Returns a list of packages that is guaranteed to be the same length as `keys`. If a package in `keys` could not be found in the store, its corresponding entry in the result will be `null`. This could be because that address never pointed to a package, or because the package was pruned.

```
multiGetPackages(
  keys: [PackageKey!]!
): [MovePackage]!
```

Arguments:

- **multiGetPackages.keys ● [PackageKey!]!** (non-null list of input object)

Type: [MovePackage]! (list of object) 24 25

multiGetTransactionEffects

Fetch transaction effects by their transactions' digests.

Returns a list of transaction effects that is guaranteed to be the same length as `keys`. If a digest in `keys` could not be found in the store, its corresponding entry in the result will be `null`. This could be because the transaction effects never existed, or because it was pruned.

```
multiGetTransactionEffects(
  keys: [String!]!
): [TransactionEffects]!
```

Arguments:

- **multiGetTransactionEffects.keys ● [String!]!** (non-null list of scalar)

Type: [TransactionEffects]! (list of object) 26 27

multiGetTransactions

Fetch transactions by their digests.

Returns a list of transactions that is guaranteed to be the same length as `keys`. If a digest in `keys` could not be found in the store, its corresponding entry in the result will be `null`. This could be because the transaction never existed, or because it was pruned.

```
multiGetTransactions(
  keys: [String!]!
): [Transaction]!
```

Arguments:

- **multiGetTransactions.keys ● [String!]!** (non-null list of scalar)

Type: [Transaction]! (list of object) 28 29

multiGetTypes

Fetch types by their string representations.

Types are canonicalized: In the input they can be at any package address at or after the package that first defines them, and in the output they will be relocated to the package that first defines them.

Returns a list of types that is guaranteed to be the same length as `keys`. If a type in `keys` could not be found, its corresponding entry in the result will be `null`.

```
multiGetTypes(
  keys: [String!]!
): [MoveType]!
```

Arguments:

- **multiGetTypes.keys ● [String!]!** (non-null list of scalar)

Type: [MoveType]! (list of object) 30 31

objectVersions

Paginate all versions of an object at `address`, optionally bounding the versions exclusively from below with `filter.afterVersion` or from above with `filter.beforeVersion`.

```
objectVersions(
  first: Int
  after: String
  last: Int
  before: String
  address: SuiAddress!
  filter: VersionFilter
): ObjectConnection
```

Arguments:

- **objectVersions.first ● Int** (scalar)
- **objectVersions.after ● String** (scalar)
- **objectVersions.last ● Int** (scalar)
- **objectVersions.before ● String** (scalar)
- **objectVersions.address ● SuiAddress!** (non-null scalar)
- **objectVersions.filter ● VersionFilter** (input object)

Type: ObjectConnection (object) 32 33

object

Fetch an object by its address.

If `version` is specified, the object will be fetched at that exact version.

If `rootVersion` is specified, the object will be fetched at the latest version at or before this version. Nested dynamic field accesses will also be subject to this bound. This can be used to fetch a child or ancestor object bounded by its root object's version. For any wrapped or child (object-owned) object, its root object can be defined recursively as:

- The root object of the object it is wrapped in, if it is wrapped.
- The root object of its owner, if it is owned by another object.
- The object itself, if it is not object-owned or wrapped.

Specifying a `version` or a `rootVersion` disables nested queries for paginating owned objects or dynamic fields (these queries are only supported at checkpoint boundaries).

If `atCheckpoint` is specified, the object will be fetched at the latest version as of this checkpoint. This will fail if the provided checkpoint is after the RPC's latest checkpoint.

If none of the above are specified, the object is fetched at the latest checkpoint.

It is an error to specify more than one of `version`, `rootVersion`, or `atCheckpoint`.

Returns `null` if an object cannot be found that meets this criteria.

```
object(
  address: SuiAddress!
  version: UInt53
  rootVersion: UInt53
  atCheckpoint: UInt53
): Object
```

Arguments:

- **object.address ● SuiAddress!** (non-null scalar)
- **object.version ● UInt53** (scalar)
- **object.rootVersion ● UInt53** (scalar)
- **object.atCheckpoint ● UInt53** (scalar)

Type: Object (object) 34 35

objects

Paginate objects in the live object set, optionally filtered by owner and/or type. The `filter` can be one of:

- A filter on type (all live objects whose type matches that filter).
- Fetching all objects owned by an address or object, optionally filtered by type.
- Fetching all shared or immutable objects, filtered by type.

```
objects(
  first: Int
  after: String
  last: Int
  before: String
  filter: ObjectFilter!
): MoveObjectConnection
```

Arguments:

- **objects.first ● Int** (scalar)
- **objects.after ● String** (scalar)
- **objects.last ● Int** (scalar)
- **objects.before ● String** (scalar)
- **objects.filter ● ObjectFilter!** (non-null input object)

Type: MoveObjectConnection (object) 36 37

packageVersions

Paginate all versions of a package at `address`, optionally bounding the versions exclusively from below with `filter.afterVersion` or from above with `filter.beforeVersion`.

Different versions of a package will have different object IDs, unless they are system packages, but will share the same original ID.

```
packageVersions(
  first: Int
  after: String
  last: Int
  before: String
  address: SuiAddress!
  filter: VersionFilter
): MovePackageConnection
```

Arguments:

- **packageVersions.first ● Int** (scalar)
- **packageVersions.after ● String** (scalar)
- **packageVersions.last ● Int** (scalar)
- **packageVersions.before ● String** (scalar)
- **packageVersions.address ● SuiAddress!** (non-null scalar)
- **packageVersions.filter ● VersionFilter** (input object)

Type: MovePackageConnection (object) 38 39

package

Fetch a package by its address.

If `version` is specified, the package loaded is the one that shares its original ID with the package at `address`, but whose version is `version`.

If `atCheckpoint` is specified, the package loaded is the one with the largest version among all packages sharing an original ID with the package at `address` and was published at or before `atCheckpoint`.

If neither are specified, the package is fetched at the latest checkpoint.

It is an error to specify both `version` and `atCheckpoint`, and `null` will be returned if the package cannot be found as of the latest checkpoint, or the address points to an object that is not a package.

Note: This interpretation of `version` and "latest" differs from the one used by `Query.object`, because non-system package upgrades generate objects with different IDs. To fetch a package using the versioning semantics of objects, use `Object.asMovePackage` nested under `Query.object`.

```
package(
  address: SuiAddress!
  version: UInt53
  atCheckpoint: UInt53
): MovePackage
```

Arguments:

- **package.address ● SuiAddress!** (non-null scalar)
- **package.version ● UInt53** (scalar)
- **package.atCheckpoint ● UInt53** (scalar)

Type: MovePackage (object) 40 41

packages

Paginate all packages published on-chain, optionally bounded to packages published strictly after `filter.afterCheckpoint` and/or strictly before `filter.beforeCheckpoint`.

```
packages(
  first: Int
  after: String
  last: Int
  before: String
  filter: PackageCheckpointFilter
): MovePackageConnection
```

Arguments:

- **packages.first ● Int** (scalar)
- **packages.after ● String** (scalar)
- **packages.last ● Int** (scalar)

- **packages.before ● String** (scalar)
- **packages.filter ● PackageCheckpointFilter** (input object)

Type: MovePackageConnection (object) [42] [43]

protocolConfigs

Fetch the protocol config by protocol version, or the latest protocol config used on chain if no version is provided.

```
protocolConfigs(
  version: UInt53
): ProtocolConfigs
```

Arguments:

- **protocolConfigs.version ● UInt53** (scalar)

Type: ProtocolConfigs (object) – Constants that control how the chain operates. These can only change during protocol upgrades which happen on epoch boundaries. Configuration is split into feature flags (booleans) and configs which can take any value (including no value at all, represented by a string) [44] .

serviceConfig

Configuration for this RPC service.

```
serviceConfig: ServiceConfig!
```

Type: ServiceConfig! (object) [45]

simulateTransaction

Simulate a transaction to preview its effects without executing it on chain.

Accepts a JSON transaction matching the **Sui gRPC API** schema. The JSON format allows for partial transaction specification where certain fields can be automatically resolved by the server.

Alternatively, for already serialized transactions, you can pass BCS-encoded data: `{"bcs": {"value": "<base64>"}}`

Unlike `executeTransaction`, this does not require signatures since the transaction is not committed to the blockchain. This allows for previewing transaction effects, estimating gas costs, and testing transaction logic without spending gas or requiring valid signatures.

```
simulateTransaction(
  transaction: JSON!
): SimulationResult!
```

Arguments:

- **simulateTransaction.transaction ● JSON!** (non-null scalar)

Type: SimulationResult! (object) 46 47

suinsName

Look-up an account by its SuiNS name, assuming it has a valid, unexpired name registration.

```
suinsName(
  address: String!
  rootVersion: UInt53
): Address
```

Arguments:

- **suinsName.address ● String!** (non-null scalar)
- **suinsName.rootVersion ● UInt53** (scalar)

Type: Address (object) 48 49

transactionEffects

Fetch transaction effects by its transaction's digest.

Returns `null` if the transaction effects do not exist in the store, either because that transaction was not executed, or it was pruned.

```
transactionEffects(
  digest: String!
): TransactionEffects
```

Arguments:

- **transactionEffects.digest ● String!** (non-null scalar)

Type: TransactionEffects (object) 50 51

transaction

Fetch a transaction by its digest.

Returns `null` if the transaction does not exist in the store, either because it never existed or because it was pruned.

```
transaction(
  digest: String!
): Transaction
```

Arguments:

- **transaction.digest ● String!** (non-null scalar)

Type: Transaction (object) 52 53

transactions

The transactions that exist in the network, optionally filtered by transaction filters.

```
transactions(
  first: Int
  after: String
  last: Int
  before: String
  filter: TransactionFilter
): TransactionConnection
```

Arguments:

- **transactions.first ● Int** (scalar)
- **transactions.after ● String** (scalar)
- **transactions.last ● Int** (scalar)
- **transactions.before ● String** (scalar)
- **transactions.filter ● TransactionFilter** (input object)

Type: TransactionConnection (object) 54 55

type

Fetch a structured representation of a concrete type, including its layout information.

Types are canonicalized: In the input they can be at any package address at or after the package that first defines them, and in the output they will be relocated to the package that first defines them.

Fails if the type is malformed, returns `null` if a type mentioned does not exist.

```
type(
  type: String!
): MoveType
```

Arguments:

- **type.type ● String!** (non-null scalar)

Type: MoveType (object) 56 57

verifyZkLoginSignature

Verify a zkLogin signature from the given `author`.

Returns a `ZkLoginVerifyResult` where `success` is `true` and `error` is empty if the signature is valid. If the signature is invalid, `success` is `false` and `error` contains the relevant error message.

- `bytes` are either the bytes of a serialized personal message, or `TransactionData`, Base64-encoded.
- `signature` is a serialized zkLogin signature, also Base64-encoded.
- `intentScope` indicates whether `bytes` are to be parsed as a personal message or `TransactionData`.
- `author` is the signer's address.

```
verifyZkLoginSignature(
  bytes: Base64!
  signature: Base64!
  intentScope: ZkLoginIntentScope!
  author: SuiAddress!
): ZkLoginVerifyResult!
```

Arguments:

- **verifyZkLoginSignature.bytes ● Base64!** (non-null scalar)
- **verifyZkLoginSignature.signature ● Base64!** (non-null scalar)
- **verifyZkLoginSignature.intentScope ● ZkLoginIntentScope!** (non-null enum)
- **verifyZkLoginSignature.author ● SuiAddress!** (non-null scalar)

Type: ZkLoginVerifyResult! (object) 58 59

Types

Directives (Schema Directives)

@deprecated

Marks an element of a GraphQL schema as no longer supported.

```
directive @deprecated(
  reason: String = "No longer supported"
) on
  | FIELD_DEFINITION
  | ARGUMENT_DEFINITION
  | INPUT_FIELD_DEFINITION
  | ENUM_VALUE
```

Arguments:

- **deprecated.reason ● String** (scalar) – Explains why this element was deprecated, usually also including a suggestion for how to access supported similar data. Formatted using the Markdown syntax, as specified by **CommonMark** 60 .

@oneOf

Indicates exactly one field must be supplied and this field must not be `null`.

```
directive @oneOf on INPUT_OBJECT
```

(No arguments for @oneOf.)

Enums

AddressTransactionRelationship

The possible relationship types for a transaction: sent or affected.

```
enum AddressTransactionRelationship {
  SENT
  AFFECTED
}
```

Values:

- **AddressTransactionRelationship.SENT** – Transactions this address has sent [61].
- **AddressTransactionRelationship.AFFECTED** – Transactions that this address was involved in, either as the sender, sponsor, or as the owner of some object that was created, modified or transferred [62].

ConsensusObjectCancellationReason

Reason why a transaction that attempted to access a consensus-managed object was cancelled.

```
enum ConsensusObjectCancellationReason {
  CANCELLED_READ
  CONGESTED
  RANDOMNESS_UNAVAILABLE
  UNKNOWN
}
```

Values:

- **CANCELLED_READ** – Read operation was cancelled [63].
- **CONGESTED** – Object congestion prevented execution [64].
- **RANDOMNESS_UNAVAILABLE** – Randomness service was unavailable [65].
- **UNKNOWN** – Internal use only [66].

ExecutionStatus

The execution status of a transaction: success or failure.

```
enum ExecutionStatus {
  SUCCESS
  FAILURE
}
```

Values:

- **SUCCESS** – The transaction was successfully executed [67].
- **FAILURE** – The transaction could not be executed [68].

MoveAbility

Abilities are keywords in Sui Move that define how types behave at the compiler level.

```
enum MoveAbility {
  COPY
  DROP
```

```
  KEY
  STORE
}
```

Values:

- **COPY** – Enables values to be copied [69].
- **DROP** – Enables values to be popped/dropped [70].
- **KEY** – Enables values to be held directly in global storage [71].
- **STORE** – Enables values to be held inside a struct in global storage [72].

MoveVisibility

The visibility modifier describes which modules can access this module member. By default, a module member can be called only within the same module.

```
enum MoveVisibility {
  PUBLIC
  PRIVATE
  FRIEND
}
```

Values:

- **PUBLIC** – A public member can be accessed by any module [73].
- **PRIVATE** – A private member can be accessed in the module it is defined in [74].
- **FRIEND** – A friend member can be accessed in the module it is defined in and any other module in its package that is explicitly specified in its friend list [75].

OwnerKind

Filter on who owns an object.

```
enum OwnerKind {
  ADDRESS
  OBJECT
  SHARED
  IMMUTABLE
}
```

Values:

- **ADDRESS** – Object is owned by an address [76].
- **OBJECT** – Object is a child of another object (e.g. a dynamic field or dynamic object field) [77].
- **SHARED** – Object is shared among multiple owners [78].

- **IMMUTABLE** – Object is frozen (immutable) [79].

RegulatedState

Whether the currency is regulated or not.

```
enum RegulatedState {
  REGULATED
  UNREGULATED
}
```

Values:

- **REGULATED** – A `DenyCap` or a `RegulatedCoinMetadata` exists for this currency [80].
- **UNREGULATED** – The currency was created without a deny list [81].

SupplyState

Future behavior of a currency's supply. (If `null`, the future behavior is not known because the currency's treasury still exists.)

```
enum SupplyState {
  BURN_ONLY
  FIXED
}
```

Values:

- **BURN_ONLY** – The supply can only decrease [82].
- **FIXED** – The supply can neither increase nor decrease [83].

TransactionKindInput

An input filter selecting for either system or programmable transactions.

```
enum TransactionKindInput {
  SYSTEM_TX
  PROGRAMMABLE_TX
}
```

Values:

- **SYSTEM_TX** – A system transaction can be one of several types of transactions (see unions/transaction-block-kind for more details) [84].

- **PROGRAMMABLE_TX** – A user submitted transaction block [85].

ZkLoginIntentScope

An enum that specifies the intent scope to be used to parse the bytes for signature verification.

```
enum ZkLoginIntentScope {
  TRANSACTION_DATA
  PERSONAL_MESSAGE
}
```

Values:

- **TRANSACTION_DATA** – Indicates that the bytes are to be parsed as transaction data bytes [86].
- **PERSONAL_MESSAGE** – Indicates that the bytes are to be parsed as a personal message [87].

Inputs (Input Object Types)

CheckpointFilter

No description.

```
input CheckpointFilter {
  afterCheckpoint: UInt53
  atCheckpoint: UInt53
  atEpoch: UInt53
  beforeCheckpoint: UInt53
}
```

Fields:

- **afterCheckpoint ● UInt53** (scalar) – Limit query results to checkpoints that occurred strictly after the given checkpoint [88].
- **atCheckpoint ● UInt53** (scalar) – Limit query results to checkpoints that occurred at the given checkpoint [89].
- **atEpoch ● UInt53** (scalar) – Limit query results to checkpoints at this epoch [90].
- **beforeCheckpoint ● UInt53** (scalar) – Limit query results to checkpoints that occurred strictly before the given checkpoint [91].

(Member Of: `Query.checkpoints` *)*

DynamicFieldName

A description of a dynamic field's name.

```
input DynamicFieldName {
  bcs: Base64!
  type: String!
}
```

Fields:

- **bcs ● Base64!** (non-null scalar) – The Base64-encoded BCS serialization of the dynamic field's "name" [92].
- **type ● String!** (non-null scalar) – The type of the dynamic field's name, like "u64" or "0x2::kiosk::Listing" [93].

EventFilter

No description.

```
input EventFilter {
  afterCheckpoint: UInt53
  atCheckpoint: UInt53
  beforeCheckpoint: UInt53
  module: String
  sender: SuiAddress
  type: String
}
```

Fields:

- **afterCheckpoint ● UInt53** (scalar) – Limit to events that occurred strictly after the given checkpoint [94].
- **atCheckpoint ● UInt53** (scalar) – Limit to events in the given checkpoint [95].
- **beforeCheckpoint ● UInt53** (scalar) – Limit to events that occurred strictly before the given checkpoint [96].
- **module ● String** (scalar) – Events emitted by a particular module. An event is emitted by a particular module if some function in the module is called by a transaction block and emits an event. Modules can be filtered by their package, or package::module. *(Filtering by emitting module and event type at the same time is not supported – providing both will cause an error.)* [97].
- **sender ● SuiAddress** (scalar) – Filter events by transaction sender address [98].
- **type ● String** (scalar) – The type of event emitted. Events can be filtered by their type's package, package::module, or fully qualified type name. (Generic types can be queried by either the generic type name, e.g. `0x2::coin::Coin`, or by the full type name such as `0x2::coin::Coin<0x2::sui::SUI>`.) [99].

(Member Of: `Query.events` *)*

ObjectFilter

A filter over the live object set. The filter can be one of:

- A filter on type (all live objects whose type matches that filter).
- Fetching all objects owned by an address or object, optionally filtered by type.
- Fetching all shared or immutable objects, filtered by type.

```
input ObjectFilter {
  owner: SuiAddress
  ownerKind: OwnerKind
  type: String
}
```

Fields:

- **owner ● SuiAddress** (scalar) – Specifies the address of the owning address or object [100]. *Required if* `ownerKind` is `"ADDRESS"` or `"OBJECT"`; if provided without `ownerKind`, then `ownerKind` defaults to `"ADDRESS"` [101].
- **ownerKind ● OwnerKind** (enum) – Filter on whether the object is address-owned, object-owned, shared, or immutable [102]. (*If set to* `"ADDRESS"` *or* `"OBJECT"`*, an* `owner` *filter must also be provided. If set to* `"SHARED"` *or* `"IMMUTABLE"`*, a* `type` *filter must also be provided.*) [103].
- **type ● String** (scalar) – Filter on the object's type [104]. The filter can be one of: a package address (e.g. `"0x2"`), a module (`"0x2::coin"`), a fully-qualified type name (`"0x2::coin::Coin"`), or a type instantiation (`"0x2::coin::Coin<0x2::sui::SUI>"`) [105].

(Member Of: `Query.objects` *)*

ObjectKey

Identifies a specific version of an object.

The `address` field must be specified, as well as at most one of `version`, `rootVersion`, or `atCheckpoint`. If none are provided, the object is fetched at the current checkpoint.

Specifying a `version` or a `rootVersion` disables nested queries for paginating owned objects or dynamic fields (these queries are only supported at checkpoint boundaries).

See `Query.object` for more details.

```
input ObjectKey {
  address: SuiAddress!
  atCheckpoint: UInt53
  rootVersion: UInt53
```

```
  version: UInt53
}
```

Fields:

- **address ● SuiAddress!** (non-null scalar) – The object's ID [106].
- **atCheckpoint ● UInt53** (scalar) – If specified, tries to fetch the latest version as of this checkpoint. Fails if the checkpoint is later than the RPC's latest checkpoint [107].
- **rootVersion ● UInt53** (scalar) – If specified, tries to fetch the latest version of the object at or before this version. Nested dynamic field accesses will also be subject to this bound [108]. *(This can be used to fetch a child or ancestor object bounded by its root object's version, as defined in Query.object's description.)* [109]
- **version ● UInt53** (scalar) – If specified, tries to fetch the object at this exact version [110].

(Member Of: `Query.multiGetObjects` *)*

PackageCheckpointFilter

Filter for paginating packages published within a range of checkpoints.

```
input PackageCheckpointFilter {
  afterCheckpoint: UInt53
  beforeCheckpoint: UInt53
}
```

Fields:

- **afterCheckpoint ● UInt53** (scalar) – Filter to packages that were published strictly after this checkpoint, defaults to fetching from the earliest checkpoint known to this RPC (this could be the genesis checkpoint, or some later checkpoint if data has been pruned) [111].
- **beforeCheckpoint ● UInt53** (scalar) – Filter to packages published strictly before this checkpoint, defaults to fetching up to the latest checkpoint (inclusive) [112].

(Member Of: `Query.packages` *)*

PackageKey

Identifies a specific version of a package.

The `address` field must be specified, as well as at most one of `version`, or `atCheckpoint`. If neither is provided, the package is fetched at the current checkpoint.

See `Query.package` for more details.

```
input PackageKey {
  address: SuiAddress!
  atCheckpoint: UInt53
  version: UInt53
}
```

Fields:

- **address ● SuiAddress!** (non-null scalar) – The object's ID (package address) [113].
- **atCheckpoint ● UInt53** (scalar) – If specified, tries to fetch the latest version as of this checkpoint [114].
- **version ● UInt53** (scalar) – If specified, tries to fetch the package at this exact version [115].

(Member Of: `Query.multiGetPackages` *)*

TransactionFilter

No description (filter for querying transactions by various criteria).

```
input TransactionFilter {
  affectedAddress: SuiAddress
  affectedObject: SuiAddress
  afterCheckpoint: UInt53
  atCheckpoint: UInt53
  beforeCheckpoint: UInt53
  function: String
  kind: TransactionKindInput
  sentAddress: SuiAddress
}
```

Fields:

- **affectedAddress ● SuiAddress** (scalar) – Limit to transactions that interacted with the given address. The address could be a sender, sponsor, or recipient of the transaction [116].
- **affectedObject ● SuiAddress** (scalar) – Limit to transactions that interacted with the given object. The object could have been created, read, modified, deleted, wrapped, or unwrapped by the transaction. *(Objects that were passed as a* `Receiving` *input are not considered to have been affected by a transaction unless they were actually received.)* [117].
- **afterCheckpoint ● UInt53** (scalar) – Filter to transactions that occurred strictly after the given checkpoint [118].
- **atCheckpoint ● UInt53** (scalar) – Filter to transactions in the given checkpoint [119].
- **beforeCheckpoint ● UInt53** (scalar) – Filter to transactions that occurred strictly before the given checkpoint [120].
- **function ● String** (scalar) – Filter transactions by Move function called. Calls can be filtered by the `package`, `package::module`, or the `package::module::name` of their function [121].

- **kind ● TransactionKindInput** (enum) – An input filter selecting for either system or programmable transactions (system vs user transactions) [122].
- **sentAddress ● SuiAddress** (scalar) – Limit to transactions that were sent by the given address [123].

(Member Of: `Query.transactions` *)*

VersionFilter

Filter for paginating the history of an Object or MovePackage.

```
input VersionFilter {
  afterVersion: UInt53
  beforeVersion: UInt53
}
```

Fields:

- **afterVersion ● UInt53** (scalar) – Filter to versions that are strictly newer than this one, defaults to fetching from the earliest version known to this RPC (this could be the initial version, or some later version if the initial version has been pruned) [124].
- **beforeVersion ● UInt53** (scalar) – Filter to versions that are strictly older than this one, defaults to fetching up to the latest version (inclusive) [125].

(Member Of: `Query.objectVersions`, `Query.packageVersions` *)*

Interfaces

IAddressable

Interface implemented by GraphQL types representing entities that are identified by an address.

An address uniquely represents either the public key of an account, or an object's ID, but never both. It is not possible to determine which type an address represents up-front. If an object is wrapped, its contents will not be accessible via its address, but it will still be possible to access other objects it owns.

```
interface IAddressable {
  address: SuiAddress!
  balance(coinType: String!): Balance
  balances(
    first: Int
    after: String
    last: Int
    before: String
  ): BalanceConnection
  defaultSuinsName: String
  multiGetBalances(keys: [String!]!): [Balance!]
```

```
  objects(
    first: Int
    after: String
    last: Int
    before: String
    filter: ObjectFilter
  ): MoveObjectConnection
}
```

Fields:

- **address ● SuiAddress!** (non-null scalar)
- **balance(coinType: String!) ● Balance** (object) – Fetch the total balance for coins of the given type (e.g. `"0x2::sui::SUI"`) owned by this address [126]. *(If no coins of that type are owned, returns a balance of zero.)*
- **balance.coinType ● String!** (non-null scalar) – *Argument specifying the coin type marker* [127].
- **balances ● BalanceConnection** (object) – Total balance across coins owned by this address, grouped by coin type [128].
- Pagination arguments: **first**, **after**, **last**, **before**.
- **defaultSuinsName ● String** (scalar) – The domain explicitly configured as the default SuiNS name for this address [129].
- **multiGetBalances(keys: [String!]!) ● [Balance!]** (list of object) – Fetch the total balances keyed by coin types (e.g. `"0x2::sui::SUI"`) owned by this address. Returns `null` when no checkpoint is set in scope (e.g. execution scope). If the address does not own any coins of a given type, a balance of zero is returned for that type [130].
- **multiGetBalances.keys ● [String!]!** (non-null list of scalar) – List of coin type markers to fetch balances for.
- **objects ● MoveObjectConnection** (object) – Objects owned by this address, optionally filtered by type [131].
- Pagination arguments: **first**, **after**, **last**, **before**; **filter** (ObjectFilter) for type filtering.

Implemented By: Address, **CoinMetadata**, **DynamicField**, **MoveObject**, **MovePackage**, **Object**, **Validator** [132].

IMoveDatatype

Interface implemented by all GraphQL types that represent a Move datatype definition (either a struct or an enum definition).

This interface provides access to fields shared by both structs and enums, e.g., the module that the datatype belongs to, the name of the datatype, type parameters, etc.

```
interface IMoveDatatype {
  abilities: [MoveAbility!]
  module: MoveModule!
  name: String!
```

```
  typeParameters: [MoveDatatypeTypeParameter!]
}
```

Fields:

- **abilities ● [MoveAbility!]** (list of enum) – Abilities on this datatype definition [133].
- **module ● MoveModule!** (non-null object) – The module that this datatype is defined in [134].
- **name ● String!** (non-null scalar) – The datatype's unqualified name [135].
- **typeParameters ● [MoveDatatypeTypeParameter!]** (list of object) – Constraints on the datatype's formal type parameters [136]. *(Note: Move bytecode does not name type parameters, so when they are referenced (e.g. in field types), they are identified by their index in this list.)*

Implemented By: MoveDatatype, **MoveEnum**, **MoveStruct** [137].

IMoveObject

Interface implemented by types that represent a Move object on-chain (a Move value whose type has `key`).

```
interface IMoveObject {
  contents: MoveValue
  dynamicField(name: DynamicFieldName!): DynamicField
  dynamicFields(
    first: Int
    after: String
    last: Int
    before: String
  ): DynamicFieldConnection
  dynamicObjectField(name: DynamicFieldName!): DynamicField
  hasPublicTransfer: Boolean
  moveObjectBcs: Base64
  multiGetDynamicFields(keys: [DynamicFieldName!]!): [DynamicField]!
  multiGetDynamicObjectFields(keys: [DynamicFieldName!]!): [DynamicField]!
}
```

Fields:

- **contents ● MoveValue** (object) – The structured representation of the object's contents [138].
- **dynamicField(name: DynamicFieldName!) ● DynamicField** (object) – Access a dynamic field on an object using its type and BCS-encoded name. Returns `null` if a dynamic field with that name could not be found attached to this object [139].
- **dynamicField.name ● DynamicFieldName!** (non-null input) – The name of the dynamic field to fetch [140].
- **dynamicFields ● DynamicFieldConnection** (object) – Dynamic fields *and dynamic object fields* owned by this object [141]. *(Dynamic fields on wrapped objects can be accessed using `Address.dynamicFields`.)*

- Pagination arguments: **first**, **after**, **last**, **before**.
- **dynamicObjectField(name: DynamicFieldName!) ● DynamicField** (object) – Access a dynamic *object* field on an object using its type and BCS-encoded name. Returns `null` if a dynamic object field with that name could not be found attached to this object [142].
- **dynamicObjectField.name ● DynamicFieldName!** (non-null input)
- **hasPublicTransfer ● Boolean** (scalar) – Whether this object can be transferred using the `TransferObjects` command or `sui::transfer::public_transfer`. *(Both require the object to have* `key` *and* `store` *abilities.)* [143].
- **moveObjectBcs ● Base64** (scalar) – The Base64-encoded BCS serialize of this object, as a `MoveObject` [144].
- **multiGetDynamicFields(keys: [DynamicFieldName!]!) ● [DynamicField]!** (non-null list of object) – Access dynamic fields on an object using their types and BCS-encoded names. Returns a list of dynamic fields the same length as `keys`. If a key could not be found, its entry will be `null` [145].
- **keys** (argument) is a list of **DynamicFieldName!**
- **multiGetDynamicObjectFields(keys: [DynamicFieldName!]!) ● [DynamicField]!** (non-null list of object) – Access dynamic object fields on an object using their types and BCS-encoded names. Returns a list of dynamic object fields the same length as `keys`. If a key could not be found, its entry will be `null` [146].

Implemented By: CoinMetadata, **DynamicField**, **MoveObject** [147].

IObject

Interface implemented by versioned on-chain values that are addressable by an ID (also referred to as its address). This includes Move objects and packages.

```
interface IObject {
  digest: String
  objectAt(version: UInt53, rootVersion: UInt53, checkpoint: UInt53): Object
  objectBcs: Base64
  objectVersionsAfter(
    first: Int
    after: String
    last: Int
    before: String
    filter: VersionFilter
  ): ObjectConnection
  objectVersionsBefore(
    first: Int
    after: String
    last: Int
    before: String
    filter: VersionFilter
  ): ObjectConnection
  owner: Owner
  previousTransaction: Transaction
  receivedTransactions(
```

```
    first: Int
    after: String
    last: Int
    before: String
    filter: TransactionFilter
  ): TransactionConnection
  storageRebate: BigInt
  version: UInt53
}
```

Fields:

- **digest ● String** (scalar) – 32-byte hash that identifies the object's contents, encoded in Base58 [148].
- **objectAt(version, rootVersion, checkpoint) ● Object** (object) – Fetch the object with the same ID, at a different version, root version bound, or checkpoint [149]. (At most one of `version`, `rootVersion`, or `checkpoint` may be specified.)
- **version ● UInt53**, **rootVersion ● UInt53**, **checkpoint ● UInt53** – optional arguments bounding the object fetch [150].
- **objectBcs ● Base64** (scalar) – The Base64-encoded BCS serialization of this object, as an `Object` [151].
- **objectVersionsAfter ● ObjectConnection** (object) – Paginate all versions of this object after this one [152].
- Pagination args: **first**, **after**, **last**, **before**; **filter** (VersionFilter).
- **objectVersionsBefore ● ObjectConnection** (object) – Paginate all versions of this object before this one [153].
- Pagination args similar to above.
- **owner ● Owner** (union) – The object's owner kind [154].
- **previousTransaction ● Transaction** (object) – The transaction that created this version of the object [155].
- **receivedTransactions ● TransactionConnection** (object) – The transactions that sent objects to this object [156].
- Pagination args: **first**, **after**, **last**, **before**; **filter** (TransactionFilter).
- **storageRebate ● BigInt** (scalar) – The MIST (micro SUI) returned to the sponsor or sender of the transaction that modifies or deletes this object [157].
- **version ● UInt53** (scalar) – The version of this object that this content comes from [158].

Implemented By: CoinMetadata, **DynamicField**, **MoveObject**, **MovePackage**, **Object** [159].

Objects (GraphQL Object Types)

(Below is an alphabetical listing of all object types in the schema, with their fields and descriptions. Many types representing connections/edges for pagination are omitted for brevity since they consist of standard fields (`edges`, `nodes`, `pageInfo`) already described under interfaces or elsewhere.)

Address

Represents an account or object address on Sui.

```
type Address implements IAddressable & IObject {
  id: ID!       # GraphQL ID (same as `address`)
  address: SuiAddress!
  # Implements IAddressable fields...
  # Implements IObject fields...
}
```

(The Address type inherits all fields of IAddressable and IObject. It primarily serves as a node with an address ID in the schema.)

Balance

Represents the balance of a particular coin type for an address.

```
type Balance {
  coinType: String!
  totalBalance: BigInt!
  objectCount: Int!
}
```

- **coinType ● String!** – The coin type marker (e.g. `"0x2::sui::SUI"`).
- **totalBalance ● BigInt!** – The total balance of that coin type.
- **objectCount ● Int!** – Number of coin objects of that type owned.

BalanceChange

Represents a change in balance (e.g. from a transaction effect).

```
type BalanceChange {
  coinType: String!
  amount: BigInt!
  owner: Owner!
}
```

- **coinType ● String!** – The coin type.
- **amount ● BigInt!** – The amount of change (positive for credit, negative for debit).
- **owner ● Owner!** – The owner of the coins for which balance changed.

BalanceConnection

A connection for grouping balances by coin type.

```
type BalanceConnection {
  edges: [BalanceEdge!]!
  nodes: [Balance!]!
  pageInfo: PageInfo!
}
```

(Each BalanceEdge contains a `node: Balance` *and* `cursor` *for pagination; nodes is the list of Balance objects.* `pageInfo` *contains pagination info.)*

Checkpoint

Checkpoints contain finalized transactions and are used for node synchronization and global transaction ordering.

```
type Checkpoint {
  digest: String!
  sequenceNumber: UInt53!
  epoch: UInt53!
  timestamp: DateTime!
  networkTotalTransactions: UInt53!
  previousDigest: String
  epochRollingGasCostSummary: GasCostSummary!
  endOfEpochData: EndOfEpochTransaction
}
```

- **digest ● String!** – The digest (hash) of this checkpoint.
- **sequenceNumber ● UInt53!** – The sequence number of this checkpoint.
- **epoch ● UInt53!** – The epoch number in which this checkpoint occurred.
- **timestamp ● DateTime!** – The timestamp of the checkpoint (in milliseconds since epoch).
- **networkTotalTransactions ● UInt53!** – Cumulative total number of transactions up to this checkpoint.
- **previousDigest ● String** – Digest of the previous checkpoint.
- **epochRollingGasCostSummary ● GasCostSummary!** – Summary of gas costs in this epoch up to this checkpoint.
- **endOfEpochData ● EndOfEpochTransaction** – If this checkpoint is the last in an epoch, this contains the end-of-epoch special transaction data (otherwise `null`).

CheckpointConnection

Connection for paginating Checkpoints.

```
type CheckpointConnection {
  edges: [CheckpointEdge!]!
  nodes: [Checkpoint!]!
```

```
  pageInfo: PageInfo!
}
```

(Edges contain `cursor` *and* `node: Checkpoint`*. Use pagination info in* `pageInfo`*.)*

CoinMetadata

An object representing metadata about a coin type (supply, symbol, decimals, etc.).

```
type CoinMetadata implements IAddressable & IMoveObject & IObject {
  id: ID!
  decimals: UInt8!
  name: String!
  symbol: String!
  description: String
  iconUrl: String
  idDomain: String
  # ...implements interfaces
}
```

- **decimals ● UInt8!** – Number of decimal places for display.
- **name ● String!** – Name of the coin.
- **symbol ● String!** – Symbol for the coin.
- **description ● String** – Description of the coin.
- **iconUrl ● String** – URL to an icon image for the coin.
- **idDomain ● String** – (If regulated) the ID domain for compliance.
- *(Implements all fields of IAddressable, IMoveObject, IObject – e.g.,* `balance`*,* `contents`*,* `owner`*, etc., specific to this coin's metadata object.)*

CoinMetadataConnection

Connection for paginating CoinMetadata objects (grouping by type, etc.).

```
type CoinMetadataConnection {
  edges: [CoinMetadataEdge!]!
  nodes: [CoinMetadata!]!
  pageInfo: PageInfo!
}
```

(Standard connection with edges and nodes of CoinMetadata.)

Command

A single command in the programmable transaction.

```
union Command = MakeMoveVecCommand
              | MergeCoinsCommand
              | MoveCallCommand
              | PublishCommand
              | SplitCoinsCommand
              | TransferObjectsCommand
              | UpgradeCommand
              | OtherCommand
```

Possible types of **Command** (each is an object type, typically with fields describing that command): - **MakeMoveVecCommand** – Create a vector (can be empty) [160] . - **MergeCoinsCommand** – Merges `coins` into the first `coin` (produces no results) [161] . - **MoveCallCommand** – *A command to execute a Move smart contract function* (no description in schema). - **PublishCommand** – Publishes a Move Package (deploy code) [162] . - **SplitCoinsCommand** – Splits off coins with specified `amounts` from a `coin`, returning multiple results [163] . - **TransferObjectsCommand** – Transfers specified object(s) to an address. All inputs must have the `store` ability and not be immutable/shared [164] . - **UpgradeCommand** – Upgrades a Move Package (to a new version) [165] . - **OtherCommand** – Placeholder for unimplemented command types [166] .

ConsensusObjectCancelled

(Object type representing a transaction that was canceled due to consensus object issues.)

```
type ConsensusObjectCancelled {
  reason: ConsensusObjectCancellationReason!
  epoch: UInt53!
  checkpoint: UInt53
}
```

- **reason ● ConsensusObjectCancellationReason!** – The reason the consensus-managed object transaction was cancelled.
- **epoch ● UInt53!** – Epoch during which it was cancelled.
- **checkpoint ● UInt53** – Checkpoint number at cancellation (if any).

DynamicField

Dynamic fields are heterogeneous fields that can be added or removed from an object at runtime.

```
type DynamicField implements IMoveObject & IObject {
  id: ID!
  name: MoveValue!
  value: MoveValue!
```

```
  # implements interfaces (owner, etc.)
}
```

- **name ● MoveValue!** – The name (key) of the dynamic field.
- **value ● MoveValue!** – The value stored in this dynamic field.
- *(Also has* `contents` *via IMoveObject, etc.)*

DynamicFieldConnection

Connection for dynamic fields of an object.

```
type DynamicFieldConnection {
  edges: [DynamicFieldEdge!]!
  nodes: [DynamicField!]!
  pageInfo: PageInfo!
}
```

(Used in `IMoveObject.dynamicFields`*.)*

DynamicFieldValue

A union representing either a dynamic field or a dynamic object field's value.

```
union DynamicFieldValue = DynamicField | DynamicField
# (in practice, dynamic object fields and regular fields are both DynamicField
objects)
```

(This union might differentiate field types if needed; currently both resolve to DynamicField.)

EndOfEpochTransaction

Represents the special transaction at the end of an epoch.

```
type EndOfEpochTransaction {
  epochId: UInt53!
  protocolVersion: UInt53!
  storageFundReinvestment: BigInt!
  storageCharge: BigInt!
  storageRebate: BigInt!
  leftOverStorageFundInflow: BigInt!
  epochStartTimestampMs: DateTime!
```

```
  epochDurationMs: UInt64!
}
```

- **epochId ● UInt53!** – The epoch number ending.
- **protocolVersion ● UInt53!** – Protocol version as of this epoch.
- **storageFundReinvestment ● BigInt!** – Amount reinvested into the storage fund.
- **storageCharge ● BigInt!** – Total storage charges.
- **storageRebate ● BigInt!** – Total storage rebates.
- **leftOverStorageFundInflow ● BigInt!** – Leftover storage fund inflow.
- **epochStartTimestampMs ● DateTime!** – Timestamp when the epoch started.
- **epochDurationMs ● UInt64!** – Duration of the epoch in milliseconds.

EndOfEpochTransactionKind

A union of all possible end-of-epoch transaction kinds.

```
union EndOfEpochTransactionKind = EndOfEpochTransaction
```

(Currently only one type, EndOfEpochTransaction, represents end-of-epoch data.)

Epoch

(See Checkpoint and EndOfEpochTransaction for details of epoch fields).

```
type Epoch {
  id: ID!                  # same as epoch sequence
  epoch: UInt53!
  systemStateVersion: UInt53!
  totalTransactions: UInt53!
  totalCheckpoints: UInt53!
  referenceGasPrice: BigInt!
  # ... other fields like protocolConfigs, validatorSet, etc.
}
```

(Fields include epoch number, system state version, counters for transactions and checkpoints, reference gas price, plus references to ProtocolConfigs, ValidatorSet, etc., which are objects containing detailed epoch info.)

EpochConnection

Connection for epochs.

```
type EpochConnection {
  edges: [EpochEdge!]!
  nodes: [Epoch!]!
```

```
  pageInfo: PageInfo!
}
```

(Edges with cursor and epoch node.)

Event

Represents an event emitted by a transaction (on-chain Move event).

```
type Event {
  id: ID!
  transaction: Transaction!
  digest: String!
  checkpoint: UInt53
  timestamp: DateTime
  sender: SuiAddress
  package: MovePackage!
  module: MoveModule!
  type: MoveType!
  parsedJson: JSON
  bcs: Base64!
}
```

- **transaction ● Transaction!** – The transaction that emitted this event.
- **digest ● String!** – Digest (ID) of the event.
- **checkpoint ● UInt53** – The checkpoint when this event was emitted.
- **timestamp ● DateTime** – Timestamp of the event.
- **sender ● SuiAddress** – Sender address of the transaction that emitted the event.
- **package ● MovePackage!** – Move package where the event's type is defined.
- **module ● MoveModule!** – Move module where the event's type is defined.
- **type ● MoveType!** – The Move type of the event (fully qualified).
- **parsedJson ● JSON** – The event data parsed as JSON, if available.
- **bcs ● Base64!** – Raw BCS-encoded bytes of the event.

EventConnection

Connection for events.

```
type EventConnection {
  edges: [EventEdge!]!
  nodes: [Event!]!
  pageInfo: PageInfo!
}
```

(Edges with event node and cursor.)

ExecutionError

Represents an error occurred during transaction execution.

```
type ExecutionError {
  code: Int!
  message: String!
}
```

- **code ● Int!** – Error code.
- **message ● String!** – Human-readable error message.

ExecutionResult

The execution result of a transaction, including the transaction effects and any potential errors due to signing or quorum-driving.

```
type ExecutionResult {
  effects: TransactionEffects
  error: ExecutionError
  timestamp: DateTime
}
```

- **effects ● TransactionEffects** – The resulting effects if execution succeeded.
- **error ● ExecutionError** – Error details if execution failed (or `null` if none).
- **timestamp ● DateTime** – Timestamp when the execution reached finality (if applicable).

FeatureFlag

Represents a feature flag in the protocol configuration (boolean feature toggle).

```
type FeatureFlag {
  name: String!
  enabled: Boolean!
}
```

- **name ● String!** – Name of the feature.
- **enabled ● Boolean!** – Whether the feature is enabled.

GasCoin

Represents a SUI Gas Coin (special coin object used for paying gas fees).

```
type GasCoin implements IMoveObject & IObject {
  id: ID!
  value: BigInt!
  # inherits MoveObject fields like hasPublicTransfer, owner, etc.
}
```

- **value ● BigInt!** – The balance (value) of this gas coin in MIST (1 SUI = 10^9 MIST).

(Implements owner, contents, etc., from IMoveObject/IObject.)

GasCostSummary

Summary of gas costs.

```
type GasCostSummary {
  computationCost: BigInt!
  storageCost: BigInt!
  storageRebate: BigInt!
  nonRefundableStorageFee: BigInt!
}
```

- **computationCost ● BigInt!** – Gas cost for computation.
- **storageCost ● BigInt!** – Gas cost for storage.
- **storageRebate ● BigInt!** – Gas rebate (returned to storage fund).
- **nonRefundableStorageFee ● BigInt!** – Non-refundable portion of storage fee.

GasEffects

Gas usage and charge effects of a transaction.

```
type GasEffects {
  gasUsed: GasCostSummary!
  objectDisposal: ObjectChange
  transitivelyClosed: Boolean!
}
```

- **gasUsed ● GasCostSummary!** – Summary of gas used.
- **objectDisposal ● ObjectChange** – An object change entry if any (like gas coin being mutated).
- **transitivelyClosed ● Boolean!** – Whether the objects in the transaction were transitively closed (no new owners created).

GasInput

Details about gas inputs to a transaction.

```
type GasInput {
  payment: CoinMetadata!
  owner: Owner!
}
```

- **payment ● CoinMetadata!** – The gas coin (or coins) used for payment.
- **owner ● Owner!** – The owner of the gas coin.

GenesisTransaction

A system transaction representing the genesis of the network (initial state).

```
type GenesisTransaction {
  objectsCreated: [ObjectChange!]!
}
```

- **objectsCreated ● [ObjectChange!]!** – List of object creations in genesis.

Immutable

(Marker object for an immutable owner state, used in Owner union.)

```
type Immutable {
  address: SuiAddress!
}
```

- **address ● SuiAddress!** – The object's address (immutable, globally accessible).

(Indicates an object is immutable and has no owner who can sign for it.)

Input (Transaction Input)

(Represents a programmable transaction input value.)

```
type Input {
  index: Int!
  type: TransactionArgument!
}
```

- **index ● Int!** – Index of the input in the transaction's input list.
- **type ● TransactionArgument!** – The type of this input (see TransactionArgument union).

Linkage

(Represents link information in Move modules, likely dependencies or similar references.)

```
type Linkage {
  type: MoveType!
  target: MoveModule!
}
```

- **type ● MoveType!** – The type that is linked.
- **target ● MoveModule!** – The module where it's linking to.

MakeMoveVecCommand, MergeCoinsCommand, MoveCallCommand, PublishCommand, SplitCoinsCommand, TransferObjectsCommand, UpgradeCommand, OtherCommand

(These are object types corresponding to each variant of the Command union. Each contains fields relevant to that specific command in a programmable transaction. For brevity, we list their main fields:)

- **MakeMoveVecCommand**: fields may include the item type and items list.
- **MergeCoinsCommand**: fields include the primary coin and list of coin IDs to merge.
- **MoveCallCommand**: fields include target function (`MoveModule` , function name, type arguments, arguments).
- **PublishCommand**: fields include modules (bytes) and dependencies.
- **SplitCoinsCommand**: fields include the source coin and split amounts.
- **TransferObjectsCommand**: fields include object IDs to transfer and the recipient address.
- **UpgradeCommand**: fields include package ID, upgrade ticket, etc.
- **OtherCommand**: (no fields; placeholder).

(Each command type has a structure reflective of the Move call or action it represents.)

MoveDatatype

Represents a Move struct type definition on-chain (fields, abilities, etc.).

```
type MoveDatatype implements IMoveDatatype {
  abilities: [MoveAbility!]!
  module: MoveModule!
  name: String!
  typeParameters: [MoveDatatypeTypeParameter!]!
  fields: [MoveField!]!
}
```

- **fields ● [MoveField!]!** – The fields (name and type) of the struct.

(Other fields from IMoveDatatype are provided via the interface.)

MoveDatatypeTypeParameter

Represents a type parameter (generic type) in a Move struct or enum definition, with its constraints.

```
type MoveDatatypeTypeParameter {
  constraints: [MoveAbility!]!
}
```

- **constraints ● [MoveAbility!]!** – The ability constraints on this type parameter (zero or more of `Copy`, `Drop`, `Key`, `Store` abilities).

MoveEnum

Represents a Move enum type definition (variants and abilities).

```
type MoveEnum implements IMoveDatatype {
  abilities: [MoveAbility!]!
  module: MoveModule!
  name: String!
  typeParameters: [MoveDatatypeTypeParameter!]!
  variants: [MoveEnumVariant!]!
}
```

- **variants ● [MoveEnumVariant!]!** – The variants of this enum (each has a name and possibly fields).

MoveEnumVariant

A variant within a Move enum type, possibly with payload fields.

```
type MoveEnumVariant {
  name: String!
  fields: [MoveField!]
}
```

- **name ● String!** – Name of the variant.
- **fields ● [MoveField!]** – Fields if this variant carries a value (similar to struct fields), or `null`/empty if no payload.

MoveField

A field of a Move struct (or an enum variant's payload).

```
type MoveField {
  name: String!
```

```
  type: MoveType!
}
```

- **name ● String!** – Field name.
- **type ● MoveType!** – Field type (fully qualified).

MoveFunction

Represents a Move function within a module (signature info).

```
type MoveFunction {
  name: String!
  visibility: MoveVisibility!
  isEntry: Boolean!
  typeParameters: [MoveFunctionTypeParameter!]!
  parameters: [MoveType!]!
  returnTypes: [MoveType!]!
}
```

- **name ● String!** – Function name.
- **visibility ● MoveVisibility!** – Visibility (public/private/friend).
- **isEntry ● Boolean!** – True if this is an entry function (callable as transaction entry).
- **typeParameters ● [MoveFunctionTypeParameter!]!** – Type parameters (generics) of the function.
- **parameters ● [MoveType!]!** – Types of function parameters.
- **returnTypes ● [MoveType!]!** – Types of return values.

MoveFunctionConnection

Connection for MoveFunction types (for modules listing functions).

```
type MoveFunctionConnection {
  edges: [MoveFunctionEdge!]!
  nodes: [MoveFunction!]!
  pageInfo: PageInfo!
}
```

MoveFunctionTypeParameter

A type parameter for a Move function, with constraints.

```
type MoveFunctionTypeParameter {
  constraints: [MoveAbility!]!
}
```

(Same as MoveDatatypeTypeParameter but in function context.)

MoveModule

Represents a Move module (collection of functions, types, etc., within a package).

```
type MoveModule {
  address: SuiAddress!
  name: String!
  fileFormatVersion: UInt8!
  friends: [MoveModule!]!
  capabilities: [FeatureFlag!]!
}
```

- **address ● SuiAddress!** – Address of the package (publisher).
- **name ● String!** – Name of the module.
- **fileFormatVersion ● UInt8!** – Move binary format version.
- **friends ● [MoveModule!]!** – Modules declared as friends (allowed to call this module's friend-only functions).
- **capabilities ● [FeatureFlag!]!** – Feature flags (capabilities) enabled in this module.

(The actual functions and types of the module are accessible via separate queries or aggregated in MovePackage info.)

MoveModuleConnection

Connection for modules in a package.

```
type MoveModuleConnection {
  edges: [MoveModuleEdge!]!
  nodes: [MoveModule!]!
  pageInfo: PageInfo!
}
```

MoveObject

A Move object (i.e., a Move struct value with key, stored on-chain).

```
type MoveObject implements IMoveObject & IObject {
  id: ID!
  type: MoveType!
  hasPublicTransfer: Boolean!
```

```
  # ... implements contents, owner, etc.
}
```

- **type ● MoveType!** – The Move type of this object (fully qualified struct type).
- **hasPublicTransfer ● Boolean!** – Whether this object can be transferred via public transfer (requires `key` and `store` abilities).

(Implements fields from interfaces, e.g., contents, dynamicFields, etc.)

MoveObjectConnection

Connection for MoveObject.

```
type MoveObjectConnection {
  edges: [MoveObjectEdge!]!
  nodes: [MoveObject!]!
  pageInfo: PageInfo!
}
```

MovePackage

A MovePackage is a kind of Object that represents code that has been published on-chain. It exposes information about its modules, type definitions, functions, and dependencies.

```
type MovePackage implements IMoveObject & IObject {
  id: ID!
  digest: String!
  # Implements IObject (owner is always immutable, etc.)
  modules: [MoveModule!]!
  linkDependencies: [MovePackage!]!
}
```

- **digest ● String!** – Package digest (hash of code).
- **modules ● [MoveModule!]!** – Modules contained in this package.
- **linkDependencies ● [MovePackage!]!** – Other packages this package depends on (by ID).

(MovePackage objects are immutable and have `owner: Immutable`.)

MovePackageConnection

Connection for MovePackage.

```
type MovePackageConnection {
  edges: [MovePackageEdge!]!
```

```
  nodes: [MovePackage!]!
  pageInfo: PageInfo!
}
```

MoveStruct

Represents an on-chain Move struct type definition (similar to MoveDatatype but specifically distinguished if needed).

```
type MoveStruct implements IMoveDatatype {
  abilities: [MoveAbility!]!
  module: MoveModule!
  name: String!
  typeParameters: [MoveDatatypeTypeParameter!]!
  fields: [MoveField!]!
}
```

(Same fields as MoveDatatype above.)

MoveType

Represents instances of concrete types (no type parameters, no references). Essentially a fully qualified Move type (struct, vector, etc.).

```
scalar MoveType
```

(MoveType values are represented as strings in fully qualified format. In schema, it's often treated as a scalar or string-like type with references to definitions.)

MoveTypeLayout

A scalar or structured representation describing the memory layout of a Move type (useful for BCS serialization).

```
scalar MoveTypeLayout
```

(Used in Move value representations; essentially an opaque descriptor of layout.)

MoveTypeSignature

A textual signature of a Move type (often identical to MoveType, fully qualified).

```
scalar MoveTypeSignature
```

(In context, likely similar to MoveType scalar usage.)

OpenMoveType

(Represents an open (generic) Move type reference, possibly including type parameters not fully specified.)

```
type OpenMoveType {
  name: String!
  module: MoveModule!
  typeParameters: [OpenMoveTypeSignature!]
}
```

(This type might represent type variables or generic placeholders.)

OpenMoveTypeSignature

A signature of an open Move type (with generics unspecialized).

```
scalar OpenMoveTypeSignature
```

Object (Generic Object)

An Object on Sui is either a typed value (a Move Object) or a Package (modules containing functions and types).

Every object on Sui is identified by a unique address, and has a version number that increases with every modification. Objects also hold metadata detailing their current owner (who can sign for access to the object and whether that access can modify and/or delete the object), and the digest of the last transaction that modified the object.

```
type Object implements IAddressable & IMoveObject & IObject {
  id: ID!
  data: MoveValue
  type: MoveType!
  owner: Owner!
  previousTransaction: Transaction!
  storageRebate: BigInt!
  # ... and all interface fields like digest, version, etc.
}
```

(The Object type provides a unified view of any on-chain object, including its Move type, BCS data, owner, etc. The `data` *field holds the contents parsed as a Move value if possible.)*

ObjectChange

Represents a change to an object as a result of a transaction (creation, mutation, deletion).

```
type ObjectChange {
  objectId: SuiAddress!
  version: UInt53
  type: MoveType
  owner: Owner
  changeType: String!
}
```

- **objectId ● SuiAddress!** – ID of the object.
- **version ● UInt53** – New version if applicable.
- **type ● MoveType** – Type of the object.
- **owner ● Owner** – New owner if changed.
- **changeType ● String!** – Kind of change ("Created", "Mutated", "Deleted", etc.).

ObjectConnection

Connection for Objects.

```
type ObjectConnection {
  edges: [ObjectEdge!]!
  nodes: [Object!]!
  pageInfo: PageInfo!
}
```

Owner (Object Owner)

(Union; see Owner union below in Unions section for details on possible owner types: AddressOwner, ObjectOwner, Shared, Immutable, ConsensusAddressOwner.)

PageInfo

Pagination information (cursor info for connections, per GraphQL cursor spec).

```
type PageInfo {
  hasNextPage: Boolean!
  hasPreviousPage: Boolean!
  startCursor: String
```

```
  endCursor: String
}
```

- **hasNextPage** / **hasPreviousPage** – Indicators for pagination.
- **startCursor** / **endCursor** – Cursors for the first and last edges in the page.

PerEpochConfig

Parameters or configuration values that are decided at epoch boundaries (like stake subsidy, etc.).

```
type PerEpochConfig {
  # fields indicating per-epoch settings...
  validatorCount: Int!
  maxValidators: Int!
  minimumStake: BigInt!
  # etc.
}
```

(Contains various fields about epoch-specific config values such as validator set sizes, stake thresholds, etc.)

ProgrammableSystemTransaction

A special Transaction type for system events that are represented as programmable transactions (likely for internal use).

```
type ProgrammableSystemTransaction {
  commands: [Command!]!
}
```

- **commands ● [Command!]!** – The sequence of command(s) in this system transaction (if any).

ProgrammableTransaction

Represents a user programmable transaction, composed of Move commands.

```
type ProgrammableTransaction {
  inputs: [TransactionInput!]!
  commands: [Command!]!
}
```

- **inputs ● [TransactionInput!]!** – Input values to the transaction (move call arguments etc.).
- **commands ● [Command!]!** – The list of commands executed.

ProtocolConfig

A single protocol configuration parameter (name and value).

```
type ProtocolConfig {
  name: String!
  value: String
}
```

- **name ● String!** – Name of the config parameter.
- **value ● String** – Value of the config parameter (as string, if set).

ProtocolConfigs

Constants that control how the chain operates (protocol versioned config).

```
type ProtocolConfigs {
  protocolVersion: UInt53!
  featureFlags: [FeatureFlag!]!
  parameters: [ProtocolConfig!]!
}
```

- **protocolVersion ● UInt53!** – The version of the protocol these configs apply to.
- **featureFlags ● [FeatureFlag!]!** – List of all feature flags and their boolean status.
- **parameters ● [ProtocolConfig!]!** – List of all configurable parameters (name-value pairs).

PublishCommand

(See Command union: PublishCommand fields correspond to publishing package bytes and dependencies.)

```
type PublishCommand {
  modules: [Base64!]!
  dependencies: [SuiAddress!]!
}
```

- **modules ● [Base64!]!** – Bytecode modules to publish (Base64-encoded).
- **dependencies ● [SuiAddress!]!** – Package addresses this package depends on (must have been published).

Pure (Command Argument for pure values)

Represents a constant or pure value used as a transaction argument.

```
type Pure {
  valueType: MoveType!
  value: JSON
}
```

- **valueType ● MoveType!** – The Move type of this pure value.
- **value ● JSON** – The actual value in JSON form (if representable as JSON).

RandomnessStateCreateTransaction / RandomnessStateUpdateTransaction

Objects representing system transactions related to randomness (perhaps VRF, etc.).

(Omitted detailed fields as they are advanced and rarely directly queried in GraphQL reference.)

Receiving

Marker type for a `Receiving` object used in Transfer commands (transferred objects).

```
type Receiving {
  objectId: SuiAddress!
}
```

- **objectId ● SuiAddress!** – ID of the object being received.

SafeMode

Information about whether an epoch was started in safe mode (some validators failed).

```
type SafeMode {
  isSafeMode: Boolean!
}
```

- **isSafeMode ● Boolean!** – True if the epoch began in safe mode.

ServiceConfig

RPC service configuration details.

```
type ServiceConfig {
  maxPageSize: Int!
  maxQueryDepth: Int!
```

```
  # ...other service limits or settings
}
```

- **maxPageSize ● Int!** – Maximum number of items per page in queries.
- **maxQueryDepth ● Int!** – Maximum GraphQL query depth allowed.

Shared / SharedInput

(Represent shared ownership in Owner union and an input type for shared objects in transactions.)

- **Shared** (owner) – see union Owner.
- **SharedInput** – likely marker indicating a Shared object being passed.

SimulationResult

The result of simulating a transaction, including the predicted effects, events, and any errors.

```
type SimulationResult {
  effects: TransactionEffects
  events: [Event!]!
  error: ExecutionError
}
```

- **effects ● TransactionEffects** – Predicted transaction effects.
- **events ● [Event!]!** – Events that would be emitted.
- **error ● ExecutionError** – Error if simulation failed.

SplitCoinsCommand, TransferObjectsCommand, UpgradeCommand, OtherCommand

(See Command union variants above; they have corresponding object types with fields capturing their specific parameters, omitted here for brevity as they mirror the variant definitions.)

StakeSubsidy

Parameters related to the stake subsidy that supplements staking rewards.

```
type StakeSubsidy {
  epochCounter: UInt53!
  balance: BigInt!
  currentEpochAmount: BigInt!
}
```

- **epochCounter ● UInt53!** – Counter of epochs elapsed for subsidy.
- **balance ● BigInt!** – Remaining subsidy fund balance.
- **currentEpochAmount ● BigInt!** – Subsidy amount for the current epoch.

StorageFund

Represents the storage fund at a point in time (SUI set aside for storage rebates).

```
type StorageFund {
  balance: BigInt!
}
```

- **balance ● BigInt!** – Amount in the storage fund.

SystemParameters

Details of system parameters decided at genesis (initial state).

```
type SystemParameters {
  maxValidators: Int!
  minValidatorStake: BigInt!
  maxValidatorStake: BigInt!
  storageGasPrice: BigInt!
}
```

(Examples of system parameters like validator counts and stake limits, gas prices, etc.)

Transaction

Description of a transaction, the unit of activity on Sui.

```
type Transaction implements IAddressable & IObject {
  id: ID!
  digest: String!
  sender: SuiAddress
  timestamp: DateTime
  checkpoint: UInt53
  transactionKind: TransactionKind!
  inputs: [TransactionInput!]!
  events: [Event!]!
  mutatedObjects: [Object!]!
  createdObjects: [Object!]!
  gasCosts: GasCostSummary!
  gasInputs: [GasInput!]!
  balanceChanges: [BalanceChange!]!
  errors: [ExecutionError!]
  # ... and interfaces fields like effects via IObject (previousTransaction not
```

```
applicable for genesis etc.)
}
```

- **digest ● String!** – Transaction digest (hash).
- **sender ● SuiAddress** – Sender address (for user transactions).
- **timestamp ● DateTime** – Time when transaction was executed.
- **checkpoint ● UInt53** – Checkpoint sequence containing this tx.
- **transactionKind ● TransactionKind!** – The kind of transaction (system vs programmable vs etc., likely a union).
- **inputs ● [TransactionInput!]!** – Inputs to the transaction (if programmable).
- **events ● [Event!]!** – Events emitted by this transaction.
- **mutatedObjects ● [Object!]!** – Objects mutated by this transaction.
- **createdObjects ● [Object!]!** – Objects created by this transaction.
- **gasCosts ● GasCostSummary!** – Gas cost summary for this tx.
- **gasInputs ● [GasInput!]!** – Gas coins and owners involved.
- **balanceChanges ● [BalanceChange!]!** – Net balance changes caused.
- **errors ● [ExecutionError!]** – Any errors encountered (if part of a programmable tx, errors might be non-fatal unless tx failed).

(Implements IAddressable so it has an `address` *field equal to some ID; implements IObject, though note transactions are ephemeral and not stored as long-lived objects except via events and digests.)*

TransactionArgument

A union of possible argument types that can be passed to Move calls in transactions.

```
union TransactionArgument = GasCoin | Input | Pure | ... (others representing object refs or values)
```

(Likely includes GasCoin for gas, Input for object reference inputs, Pure for pure values, etc.)

TransactionConnection

Connection for Transaction objects.

```
type TransactionConnection {
  edges: [TransactionEdge!]!
  nodes: [Transaction!]!
  pageInfo: PageInfo!
}
```

TransactionEffects

The results of executing a transaction.

```
type TransactionEffects {
  status: ExecutionStatus!
  mutated: [ObjectChange!]!
  created: [ObjectChange!]!
  deleted: [ObjectChange!]!
  gasUsed: GasCostSummary!
}
```

- **status ● ExecutionStatus!** – SUCCESS or FAILURE.
- **mutated ● [ObjectChange!]!** – List of objects mutated.
- **created ● [ObjectChange!]!** – List of objects created.
- **deleted ● [ObjectChange!]!** – List of objects deleted.
- **gasUsed ● GasCostSummary!** – Gas used for the transaction.

TransactionInput

An input value provided to a programmable transaction (could be an object or pure value).

```
type TransactionInput {
  index: Int!
  type: TransactionArgument!
}
```

- **index ● Int!** – The index/order of this input.
- **type ● TransactionArgument!** – The argument (object or pure) that this input represents.

TransactionKind

Union of the different kinds of transactions (system transactions vs ProgrammableTransaction, etc.).

```
union TransactionKind = ProgrammableTransaction
                      | GenesisTransaction
                      | ConsensusCommitPrologueTransaction
                      | ChangeEpochTransaction
                      | ...
```

(Likely includes all variants such as Genesis, Consensus prologue, Epoch change, Programmable, etc.)

UnchangedConsensusObject

A union or object indicating a consensus-managed object that did not change state? Possibly a placeholder.

(Omitted details as it might correspond to a union for consensus object state not changing.)

Validator

Represents a validator node's on-chain state.

```
type Validator implements IAddressable & IObject {
  id: ID!
  metadata: ValidatorMetadata!
  stake: BigInt!
  pendingStake: BigInt!
  commissionRate: Int!
  # ... more fields like voting power, gas info
}
```

(Contains fields like stake amounts, metadata (name, pubkey, network addresses, etc.), commission, etc., representing a validator's info in the current epoch.)

ValidatorAggregatedSignature

Represents an aggregated signature from validators (e.g. committee signature on checkpoint or epoch).

```
type ValidatorAggregatedSignature {
  signature: Base64!
  bitmap: Base64!
}
```

- **signature ● Base64!** – Aggregated signature bytes.
- **bitmap ● Base64!** – Bitmask indicating which validators contributed.

ValidatorConnection, ValidatorEdge, ValidatorSet

These types represent the set of validators and allow pagination if needed.

- **ValidatorSet**: an object containing current and next epoch validator info.
- **ValidatorConnection**, **ValidatorEdge**: for listing validators.

(The ValidatorSet likely contains fields like active validator list, pending changes, etc.)

ZkLoginVerifyResult

The result of the zkLogin signature verification.

```
type ZkLoginVerifyResult {
  success: Boolean!
```

```
  error: String
}
```

- **success ● Boolean!** – True if signature is valid.
- **error ● String** – Error message if not valid (null if success).

Scalars

Base64

String containing Base64-encoded binary data.

```
scalar Base64
```

(Used for representing raw binary data encoded in Base64.)

BigInt

String representation of an arbitrary width, possibly signed integer.

```
scalar BigInt
```

(Used for 64-bit or larger integers, such as balances, that can exceed 32-bit range.)

Boolean

The `Boolean` scalar type represents `true` or `false`.

```
scalar Boolean
```

(Standard GraphQL boolean type.)

DateTime

Date and time, represented as a timestamp (generally in milliseconds since Unix epoch) encoded as a 64-bit integer.

```
scalar DateTime
```

(Often appears as an ISO 8601 string in responses.)

Int

The `Int` scalar type represents non-fractional signed whole numeric values. Int can represent values between -(2^31) and 2^31 - 1.

```
scalar Int
```

(Standard 32-bit integer.)

JSON

Arbitrary JSON data. The `JSON` scalar type allows representation of JSON values (objects, arrays, numbers, strings, booleans, or null).

```
scalar JSON
```

(Used for e.g. event `parsedJson` *field or for passing in transaction JSON structures.)*

MoveTypeLayout

Opaque scalar describing the binary layout of a Move type.

```
scalar MoveTypeLayout
```

(Advanced; used in Move value representation.)

MoveTypeSignature

Opaque scalar representing the type signature string of a Move value or type.

```
scalar MoveTypeSignature
```

OpenMoveTypeSignature

Scalar for an open (unresolved generic) Move type signature.

```
scalar OpenMoveTypeSignature
```

String

The `String` scalar type represents textual data, represented as UTF-8 character sequences. The String type is most often used by GraphQL to represent free-form human-readable text.

```
scalar String
```

(Standard GraphQL string type.)

SuiAddress

A Sui address (an account or object ID), represented as a 32-byte hexadecimal string (prefixed by "0x").

```
scalar SuiAddress
```

(Used for account addresses and object IDs in the API.)

UInt53

An unsigned 53-bit integer (the maximum integer safely representable in JavaScript).

```
scalar UInt53
```

(Used for sequence numbers, epoch numbers, etc., which can be up to 2^53 - 1.)

Unions

Command (union)

See **Command** under Objects above – a union of all programmable transaction command types (MakeMoveVecCommand, MergeCoinsCommand, etc.) [167] .

DynamicFieldValue (union)

Union covering dynamic field vs dynamic object field values (currently effectively both **DynamicField**) [168] .

EndOfEpochTransactionKind (union)

Union of possible end-of-epoch transaction kinds (currently just **EndOfEpochTransaction**) [169] .

Owner (union)

Union of types describing an object's ownership:

```
union Owner = AddressOwner | ObjectOwner | Shared | Immutable | ConsensusAddressOwner
```

- **AddressOwner** – Object is exclusively owned by a single address (mutable) [170].
- **ObjectOwner** – Object is exclusively owned by a single object (mutable). *The owning object may be inaccessible if wrapped.* [171].
- **Shared** – Object is shared (globally accessible, mutable) [172].
- **Immutable** – Object is immutable (globally accessible, cannot be modified) [173].
- **ConsensusAddressOwner** – Object is exclusively owned by a single address and its usage is gated by consensus (e.g., a special system object) [174].

TransactionArgument (union)

Union of GraphQL types that can appear as transaction arguments:

```
union TransactionArgument = GasCoin | Input | Pure | SharedInput | ...
```

(Covers GasCoin references, object Input references, Pure values, possibly others like Move package input, etc. In schema references, appears as field types in `TransactionInput` *or* `ProgrammableTransaction`*.)*

TransactionInput (union)

(Potentially a union similar to TransactionArgument; the schema lists TransactionInput as an object type already.)

(There is also TransactionKind union, listing different transaction variants: ProgrammableTransaction, GenesisTransaction, ConsensusCommitPrologueTransaction, ChangeEpochTransaction, etc., as indicated earlier.)

UnchangedConsensusObject (union)

Union for consensus-managed objects that remain unchanged in a transaction (perhaps not explicitly used in queries directly).

```
union UnchangedConsensusObject = ConsensusObjectRead | ...
```

(Likely includes ConsensusObjectRead and similar types for when consensus object access yields no state change.)

---

1 2 executeTransaction | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/mutations/execute-transaction

3 4 address | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/address

5 chainIdentifier | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/chain-identifier

6 7 checkpoint | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/checkpoint

8 9 checkpoints | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/checkpoints

10 11 coinMetadata | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/coin-metadata

12 13 epoch | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/epoch

14 15 epochs | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/epochs

16 17 events | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/events

18 19 multiGetCheckpoints | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-checkpoints

20 21 multiGetEpochs | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-epochs

22 23 multiGetObjects | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-objects

24 25 multiGetPackages | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-packages

26 27 multiGetTransactionEffects | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-transaction-effects

28 29 multiGetTransactions | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-transactions

30 31 multiGetTypes | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/multi-get-types

32 33 objectVersions | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/object-versions

34 35 object | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/object

36 37 objects | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/objects

38 39 packageVersions | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/package-versions

40 41 package | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/package

42 43 packages | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/packages

44 protocolConfigs | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/protocol-configs

45 serviceConfig | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/service-config

46 47 simulateTransaction | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/simulate-transaction

48 49 suinsName | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/suins-name

50 51 transactionEffects | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/transaction-effects

52 53 transaction | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/transaction

54 55 transactions | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/transactions

56 57 type | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/type

58 59 verifyZkLoginSignature | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/operations/queries/verify-zk-login-signature

60 deprecated | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/directives/deprecated

61 62 AddressTransactionRelationship | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/address-transaction-relationship

63 64 65 66 ConsensusObjectCancellationReason | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/consensus-object-cancellation-reason

67 68 ExecutionStatus | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/execution-status

69 70 71 72 MoveAbility | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/move-ability

73 74 75 MoveVisibility | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/move-visibility

76 77 78 79 OwnerKind | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/owner-kind

80 81 RegulatedState | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/regulated-state

82 83 SupplyState | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/supply-state

84 85 TransactionKindInput | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/transaction-kind-input

86 87 ZkLoginIntentScope | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/enums/zk-login-intent-scope

88 89 90 91 CheckpointFilter | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/checkpoint-filter

92 93 DynamicFieldName | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/dynamic-field-name

94 95 96 97 98 99 EventFilter | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/event-filter

100 101 102 103 104 105 ObjectFilter | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/object-filter

106 107 108 109 110 ObjectKey | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/object-key

111 112 PackageCheckpointFilter | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/package-checkpoint-filter

113 114 115 PackageKey | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/package-key

116 117 118 119 120 121 122 123 TransactionFilter | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/transaction-filter

124 125 VersionFilter | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/inputs/version-filter

126 127 128 129 130 131 132 IAddressable | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/interfaces/iaddressable

133 134 135 136 137 IMoveDatatype | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/interfaces/imove-datatype

138 139 140 141 142 143 144 145 146 147 IMoveObject | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/interfaces/imove-object

148 149 150 151 152 153 154 155 156 157 158 159 IObject | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/interfaces/iobject

160 161 162 163 164 165 166 167 168 Command | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/unions/command

169 170 171 172 173 174 Owner | Sui Documentation
https://docs.sui.io/references/sui-api/sui-graphql/beta/reference/types/unions/owner